CS34020/CSM4220 Workshop 5

Image matching

Theme: Smart Cities

Purpose and theme: in this workshop we will try out some methods for matching points and objects between images and using them in applications such as template matching and simultaneous localisation and mapping (SLAM). The tasks in this workshop are loosely around the theme of *smart cities* but could be applied in other domains.

Part 1: Spotting Road Signs

First, we're going to use some basic template matching to see how that can work. The following code has been adapted from the OpenCV Template Matching tutorial (https://docs.opencv.org/4.x/d4/dc6/tutorial_py_template_matching.html).

A subset of images from **Road Sign Detection**[1] dataset has been supplied. Start with this code, and add in parts where noted:

```
# ADD CODE HERE: import opencv and numpy

from matplotlib import pyplot as plt

# ADD CODE HERE: read road529.png in greyscale, call it 'img'
# ADD CODE HERE: read 30sign.png in greyscale, call it 'template'

w, h = template.shape[::-1] # get the size of the template image
# Apply template Matching
result = cv.matchTemplate(img, template, cv.TM_CCOEFF)

# get the values and locations of the minimum and maximum
# values from the result so we know where to draw the bounding box
min_val, max_val, min_loc, max_loc = cv.minMaxLoc(result)
top_left = max_loc
bottom_right = (top_left[0] + w, top_left[1] + h)

# draw the box on the image and plot it
cv.rectangle(img,top_left, bottom_right, 255, 2)
plt.imshow(img, cmap = 'gray') # what to plot and colour map to use
plt.xticks([]), plt.yticks([]) # hides the plot axes
plt.show()
```

[1] https://www.kaggle.com/datasets/andrewmvd/road-sign-detection/

The template matching takes the image and the template to match as the first two arguments. The third is the method used for matching (we saw some of these in the lectures); there are six methods[2] available:

- TM_SQDIFF
- TM_SQDIFF_NORMED
- TM_CCORR
- TM_CCORR_NORMED
- TM_CCOEFF
- TM_CCOEFF_NORMED

We're going to try each, but you should see very little difference in the output image, so we are going to look at the raw match results too. You already have this data – the template matching has put it into a variable called **result**. Try showing the result instead of **img**, and see how the different matching techniques find the match.

Try replacing the image road529.png with road528.png – an image of the same sign from a slightly different perspective. You should see that this method is not robust to variation. Let's try something that is …

Create a new file which starts the same way as the previous one: import opencv, numpy, and matplotlib, then read in road529.png and 30sign.png. Next add this, adapted from the Feature Matching Tutorial (https://docs.opencv.org/3.4/dc/dc3/tutorial_py_matcher.html):

```
# Initiate ORB detector
orb = cv.ORB_create()

# find the keypoints and descriptors with ORB
kp1, des1 = orb.detectAndCompute(img,None)
kp2, des2 = orb.detectAndCompute(template,None)

# create BFMatcher object
bf = cv.BFMatcher(cv.NORM_HAMMING, crossCheck=True)

# Match descriptors
matches = bf.match(des1,des2)

# Sort them in the order of their distance
matches = sorted(matches, key = lambda x:x.distance)

# Draw first 10 matches
img3 = cv.drawMatches(img, kp1, template, kp2, matches[:10], None,
                      flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
plt.imshow(img3)
```

[2] Read more about template matching here: https://docs.opencv.org/4.x/df/d@/group__imgproc__object.html

```
plt.show()
```

It should find the matching features quite nicely! Experiment with different images of roads and signs – some will work much better than others, can you see why?

Part 2: Visual SLAM

Simultaneous localisation and mapping (SLAM) is important in robotics and vehicle navigation. It allows a mobile robot or autonomous car to build up a map of it's environment (e.g. as a set of points) while simultaneously estimating it's location and orientation within that map. Different sensors can be used e.g. range finders, Lidar etc., but the version that uses a camera feed is known as Visual-SLAM. When only a single camera is involved it is Monocular Visual SLAM, but it can also be implemented to use multiple cameras. We look at a monocular version of Visual SLAM.

SLAM has a number of "front end" and "back end" tasks. Front end tasks are focussed on creating a local map and estimating the position and orientation within it, steps include detecting feature points, matching the feature points, pose estimation, triangulation and creating a local map. Back-end tasks are designed to improve the basic local maps and deal with "drift" as the camera moves further from the start point. Back-end steps include bundle adjustment (to simultaneously refine the camera poses and points so they match consistently), loop closure (to detect when the camera returns to an earlier location and use that to improve the map), pose graph optimisation (to update the whole map and path when a loop is closed).

Here, we will mostly be focussing on the front-end tasks. If you are interested in reading more there is a nice tutorial on learnopencv.com[3], where the example code was adapted from. We will use the ORB detector again to detect and match features. You are provided with an outline implementation in the file **vslam.py**, you just need to add some code to extract points for matching using ORB features (as above) and use *ransac* to select points using the epipolar constraint. First use ORB to detect features in the extract function, here:

```
def extract(img):
     orb = cv2.ORB_create()
     kps, des = orb.detectAndCompute(img,None)

     return np.array([(kp.pt[0], kp.pt[1]) for kp in kps]), des
```

Next in the match_frames function, use the detected points and their descriptors (in f1.des and f2.des) to find the initial correspondences:

```
def match_frames(f1, f2):
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
    matches = bf.match(f1.des, f2.des)
```

The provided code does some additional checks that the points are quite close, in normalised image coordinates, assuming that the distance between points should be less than 0.1 in this space. Once a set of good matches are found, they can be used with RANSAC to find the fundamental matrix, and the provided extractPose function to calculate the pose matrix from that:

```
   # Fit matrix
    model, inliers = cv2.findFundamentalMat(ret[:,0], ret[:,1],
                                cv2.FM_RANSAC, 0.005, 0.99, 200)
    inliers = (inliers.ravel() == 1)

    # Ignore outliers
    ret = ret[inliers]
    Rt = extractPose(model)

    return idx1[inliers], idx2[inliers], Rt
```

Complete the provided code using the hints given above to replace the `#TODO` comments. Before running the code, I found it necessary to use the "magic" command in the python terminal in Spyder, in order to draw the matplotlib diagrams in a pop-up window (otherwise it doesn't update for some reason):

```
%matplotlib auto
```

The visualisation of the map points (in blue) and the estimated location (in red) can get rather slow after a few frames due to the number of points.

[3] https://learnopencv.com/monocular-slam-in-python/

You could try out different options and parameter settings, for example the original *learnopencv* code used "*goodFeaturesToTrack*" rather than ORB features (but still used ORB feature descriptors to match). Does the cross checking (crossCheck=True) in the *BFMatcher* help or not? The original code matched to the 2 nearest neighbours (using knnMatch) and made sure that the closest match was less than 75% of the distance to the second-best match. This is known as Lowe's ratio test, which tries to only select best match points that are far from other matching points, as this means they are more reliable. You could try putting this extra check in.

Link to the assignment

In the assignment you are asked to detect yellow cars. Consider the following questions relating to the assignment, and **write down your answers**, you will need them for the assignment report.

Questions for CS34020 and CSM4220 Students

Write a short paragraph discussing how Visual-SLAM could be combined with yellow car detection and tracking to allow an agent (e.g. a robot) to locate the 3D position of the yellow cars in relation to the agent. You will be asked to include this in your assignment submission.

Submission

Please **submit your work from this workshop**, to Blackboard at the end of the practical. The submissions themselves are not assessed and carry no marks, but will be used as an additional measure of **engagement**. Don't worry if you haven't completed all aspects of this worksheet, just submit what you have done.